

MODUL 5

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: // Ini komentar

- Banyak baris: /* Ini komentar yang panjang */

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- main() : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. std adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisa bagi).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI LINKED LIST

Singly Linked List adalah struktur data yang fleksibel dan efisien untuk banyak aplikasi. Memahami dasar teori tentang Singly Linked List sangat penting untuk memilih struktur data yang tepat untuk masalah tertentu dan untuk mengimplementasikan algoritma yang efisien.

Definisi:

- Linked List (Daftar Berantai): Struktur data linear di mana elemen-elemen data (disebut node) disusun secara sekuensial, dengan setiap node berisi data dan pointer (penunjuk) ke node berikutnya dalam urutan.

- Singly Linked List (Daftar Berantai Tunggal): Jenis linked list di mana setiap node hanya memiliki satu pointer, yaitu pointer ke node berikutnya. Ini memungkinkan traversal (penelusuran) list hanya dalam satu arah (dari awal ke akhir).

Komponen Utama:

1. Node (Simpul):

- Unit dasar dalam linked list.

- Setiap node berisi dua bagian:

- Data: Informasi yang disimpan dalam node (bisa berupa tipe data apa saja, seperti integer, string, objek, dll.).

- Next (Berikutnya): Pointer yang menunjuk ke node berikutnya dalam list. Jika node ini adalah node terakhir, pointer next akan berisi nilai NULL (atau nullptr dalam C++11 ke atas).

2. Head (Kepala):

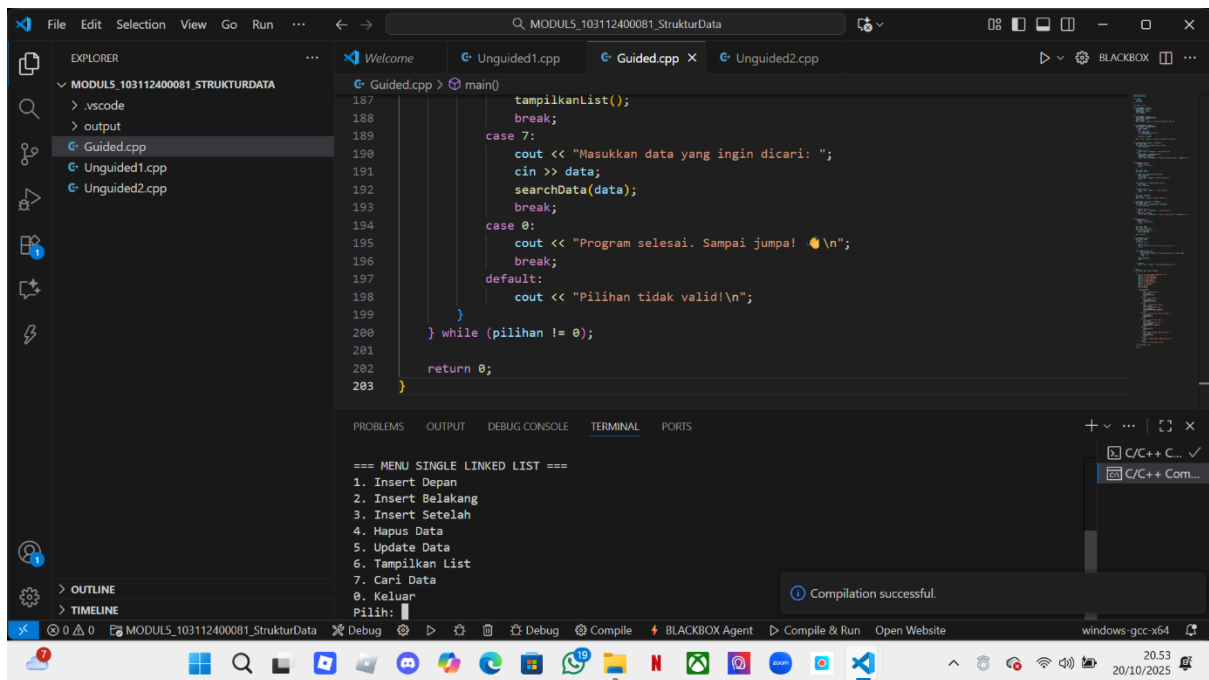
- Pointer yang menunjuk ke node pertama dalam list.

- Jika list kosong, pointer head akan berisi nilai NULL .

Searching

Searching merupakan operasi dasar list dengan melakukan aktivitas pencarian terhadap node tertentu. Proses ini berjalan dengan mengunjungi setiap node dan berhenti setelah node yang dicari ketemu. Dengan melakukan operasi searching, operasi-operasi seperti insert after, delete after, dan update akan lebih mudah. Semua fungsi dasar di atas merupakan bagian dari ADT dari singgle linked list, dan aplikasi pada bahasa pemrograman C++ semua ADT tersebut tersimpan dalam file *.c dan file *.h.

Guided 1



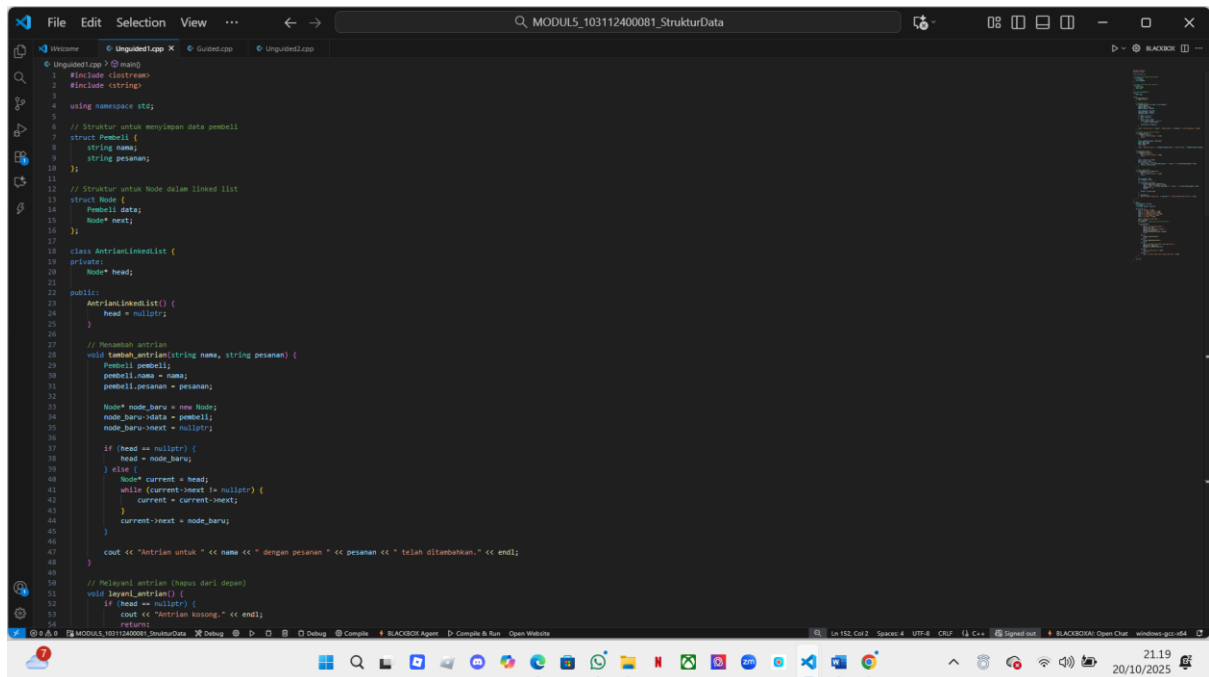
```
187     tampilkanList();
188     break;
189     case 7:
190     cout << "Masukkan data yang ingin dicari: ";
191     cin >> data;
192     searchData(data);
193     break;
194     case 0:
195     cout << "Program selesai. Sampai jumpa! 🌟\n";
196     break;
197     default:
198     cout << "Pilihan tidak valid!\n";
199     }
200     } while (pilihan != 0);
201
202     return 0;
203 }
```

=== MENU SINGLE LINKED LIST ===
1. Insert Depan
2. Insert Belakang
3. Insert Setelah
4. Hapus Data
5. Update Data
6. Tampilkan List
7. Cari Data
0. Keluar
Pilih: 0

Compilation successful.

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita melakukan manipulasi dasar pada sebuah single linked list. Coding bisa dirunning namun outputnya selalu geser ke atas dan kebawah, saya tidak tahu apa masalahnya.

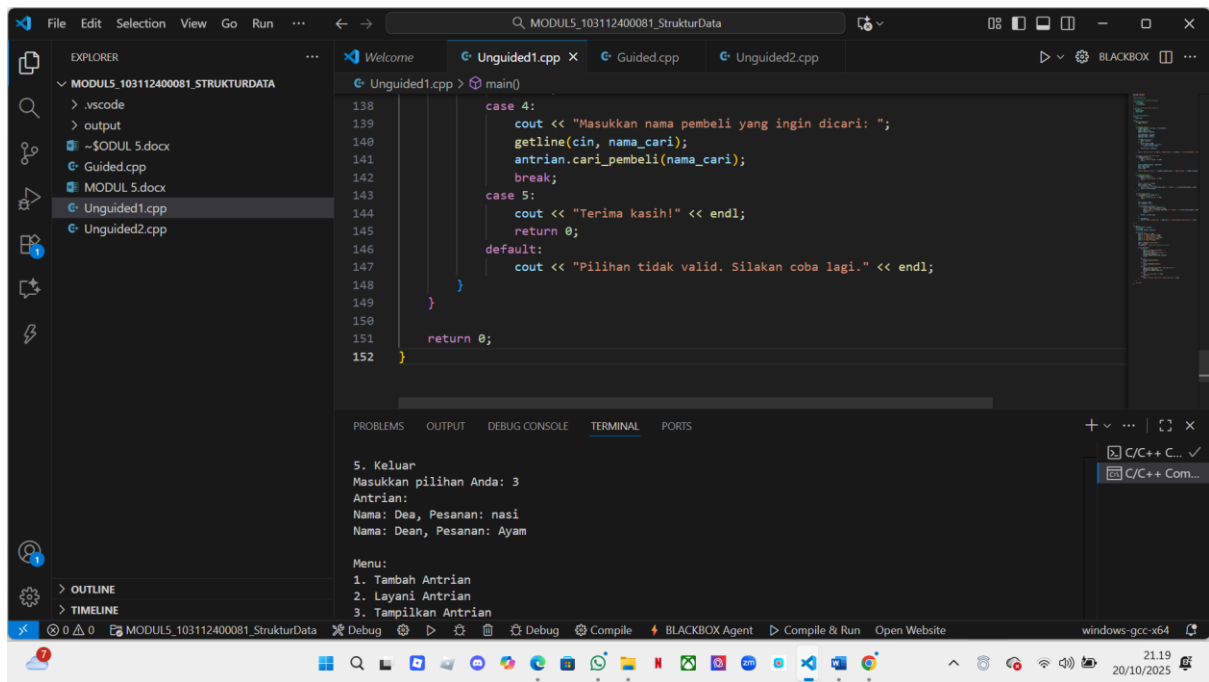
Unguided 1



```
1 // Struktur untuk menyimpan data pembell
2 struct Pembell {
3     string nama;
4     string pesanan;
5 };
6
7 // Struktur untuk Node dalam linked list
8 struct Node {
9     Pembell data;
10    Node* next;
11 };
12
13 class AntrianLinkedList {
14 private:
15     Node* head;
16 public:
17     AntrianLinkedList() {
18         head = nullptr;
19     }
20
21     // Penambahan antrian
22     void tambah_antrian(string nama, string pesanan) {
23         Pembell pembell;
24         pembell.nama = nama;
25         pembell.pesanan = pesanan;
26
27         Node* node_baru = new Node;
28         node_baru->data = pembell;
29         node_baru->next = nullptr;
30
31         if (head == nullptr) {
32             head = node_baru;
33         } else {
34             Node* current = head;
35             while (current->next != nullptr) {
36                 current = current->next;
37             }
38             current->next = node_baru;
39         }
40
41         cout << "Antrian untuk " << nama << " dengan pesanan " << pesanan << " telah ditambahkan." << endl;
42     }
43
44     // Hapus antrian (hapus dari depan)
45     void hapus_antrian() {
46         if (head == nullptr) {
47             cout << "Antrian kosong." << endl;
48             return;
49         }
50         head = head->next;
51     }
52
53     // Tampilkan antrian
54     void tampilkan_antrian() {
55         if (head == nullptr) {
56             cout << "Antrian kosong." << endl;
57             return;
58         }
59         Node* current = head;
60         while (current != nullptr) {
61             cout << "Nama: " << current->data.nama << ", Pesanan: " << current->data.pesanan << ", ";
62             current = current->next;
63         }
64         cout << endl;
65     }
66 }
```

```
File Edit Selection View ... MODUL5_103112400081_StrukturData
Unquoted.cpp x Guided.cpp Unquoted.cpp
class AntrianLinkedist {
    void layan_antrian() {
        if (head == nullptr) {
            return;
        }
        Pembeli pembeli_dilayani = head->data;
        Node* temp = head;
        head = head->next;
        delete temp;
        cout << "Dilayani antrian: " << pembeli_dilayani.nama << " dengan pesanan " << pembeli_dilayani.pesanan << endl;
    }
    // Menampilkan antrian
    void tampilkan_antrian() {
        if (head == nullptr) {
            cout << "Antrian kosong." << endl;
            return;
        }
        cout << "Antrian:" << endl;
        Node* current = head;
        while (current != nullptr) {
            cout << "Nama: " << current->data.nama << ", Pesanan: " << current->data.pesanan << endl;
            current = current->next;
        }
    }
    // Pencari nama pembeli
    void cari_pembeli(string nama_cari) {
        if (head == nullptr) {
            cout << "Antrian kosong." << endl;
            return;
        }
        Node* current = head;
        bool ditemukan = false;
        while (current != nullptr) {
            if (current->data.nama == nama_cari) {
                cout << "Pembeli ditemukan:" << endl;
                cout << "Nama: " << current->data.nama << ", Pesanan: " << current->data.pesanan << endl;
                ditemukan = true;
                break;
            }
            current = current->next;
        }
        if (!ditemukan) {
            cout << "Pembeli dengan nama " << nama_cari << " tidak ditemukan dalam antrian." << endl;
        }
    }
};
```

```
File Edit Selection View ... MODUL5_103112400081_StrukturData
Unquoted.cpp x Guided.cpp Unquoted.cpp
class AntrianLinkedist {
    void cari_pembeli(string nama_cari) {
    };
};
int main() {
    AntrianLinkedist antrian;
    int pilihan;
    string nama, pesanan, nama_cari;
    while (true) {
        cout << "Menu:" << endl;
        cout << "1. Tambah Antrian" << endl;
        cout << "2. Layani Antrian" << endl;
        cout << "3. Tampilkan Antrian" << endl;
        cout << "4. Cari Pembeli" << endl;
        cout << "5. Keluar" << endl;
        cout << "Masukkan pilihan Anda: ";
        cin >> pilihan;
        cin.ignore(); // Memerihkan newline dari buffer
        switch (pilihan) {
            case 1: {
                cout << "Masukkan nama pembeli: ";
                getline(cin, nama);
                cout << "Masukkan pesanan pembeli: ";
                getline(cin, pesanan);
                antrian.tambah_antrian(nama, pesanan);
                break;
            }
            case 2: {
                antrian.layan_antrian();
                break;
            }
            case 3: {
                antrian.tampilkan_antrian();
                break;
            }
            case 4: {
                cout << "Masukkan nama pembeli yang ingin dicari: ";
                getline(cin, nama_cari);
                antrian.cari_pembeli(nama_cari);
                break;
            }
            case 5: {
                cout << "Terima kasih!" << endl;
                return 0;
            }
            default: {
                cout << "Pilihan tidak valid. Silakan coba lagi." << endl;
            }
        }
    }
    return 0;
}
```

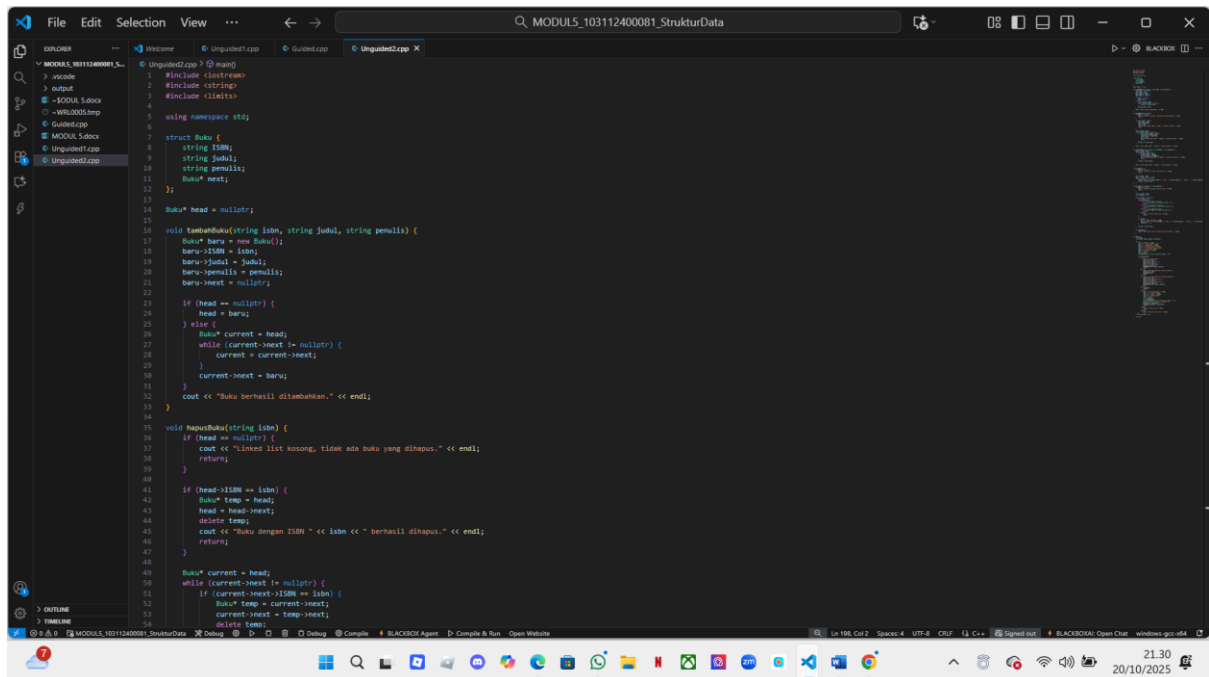
```
138         case 4:
139             cout << "Masukkan nama pembeli yang ingin dicari: ";
140             getline(cin, nama_cari);
141             antrian.cari_pembeli(nama_cari);
142             break;
143         case 5:
144             cout << "Terima kasih!" << endl;
145             return 0;
146         default:
147             cout << "Pilihan tidak valid. Silakan coba lagi." << endl;
148     }
149 }
150
151 return 0;
152 }
```

5. Keluar
Masukkan pilihan Anda: 3
Antrian:
Nama: Dea, Pesanan: nasi
Nama: Dean, Pesanan: Ayam

Menu:
1. Tambah Antrian
2. Layani Antrian
3. Tampilkan Antrian

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membuat program untuk mengelola antrian pembeli. Dalam konteks ini Searching Linked Listnya itu seperti yang ada di bagian fungsi “**Cari Pembeli**”. Yang mana memungkinkan program untuk menemukan dan menampilkan informasi pembeli tertentu berdasarkan nama mereka dalam antrian.

Unguided 2



```
17 struct Buku {
18     string ISBN;
19     string judul;
20     string penulis;
21     Buku* next;
22 };
23
24 Buku* head = nullptr;
25
26 void tambahBuku(string isbn, string judul, string penulis) {
27     Buku* baru = new Buku();
28     baru->ISBN = isbn;
29     baru->judul = judul;
30     baru->penulis = penulis;
31     baru->next = nullptr;
32
33     if (head == nullptr) {
34         head = baru;
35     } else {
36         Buku* current = head;
37         while (current->next != nullptr) {
38             current = current->next;
39         }
40         current->next = baru;
41     }
42     cout << "Buku berhasil ditambahkan." << endl;
43 }
44
45 void hapusBuku(string isbn) {
46     if (head == nullptr) {
47         cout << "Linked list kosong, tidak ada buku yang dihapus." << endl;
48         return;
49     }
50     if (head->ISBN == isbn) {
51         Buku* temp = head;
52         head = head->next;
53         delete temp;
54         cout << "Buku dengan ISBN " << isbn << " berhasil dihapus." << endl;
55         return;
56     }
57     Buku* current = head;
58     while (current->next != nullptr) {
59         if (current->next->ISBN == isbn) {
60             Buku* temp = current->next;
61             current->next = temp->next;
62             delete temp;
63             cout << "Buku dengan ISBN " << isbn << " berhasil dihapus." << endl;
64             return;
65         }
66         current = current->next;
67     }
68     cout << "Buku dengan ISBN " << isbn << " tidak ditemukan." << endl;
69 }
```

```
File Edit Selection View ...
MODUL5_10311240001_StrukturData
> include
> output
> -MODUL5_Solusi
> -MODUL5_Solusi
> Modul5.cpp
> MODUL5_Solusi
> Unguiled1.cpp
> Unguiled2.cpp
> Unguiled3.cpp

60 Buku* current = head;
61 while (current != nullptr) {
62     if (current->ISBN == isbn) {
63         Buku* temp = current->next;
64         current->next = temp->next;
65         delete temp;
66         cout << "Buku dengan ISBN " << isbn << " berhasil dihapus." << endl;
67         return;
68     }
69     current = current->next;
70 }
71 cout << "Buku dengan ISBN " << isbn << " tidak ditemukan." << endl;
72 }
73
74 void perbaruiBuku(string isbn, string newJudul, string newPenulis) {
75     Buku* current = head;
76     while (current != nullptr) {
77         if (current->ISBN == isbn) {
78             current->Judul = newJudul;
79             current->Penulis = newPenulis;
80             cout << "Buku dengan ISBN " << isbn << " berhasil diperbarui." << endl;
81             return;
82         }
83         current = current->next;
84     }
85     cout << "Buku dengan ISBN " << isbn << " tidak ditemukan." << endl;
86 }
87
88 void lihatBuku() {
89     if (head == nullptr) {
90         cout << "Tidak ada buku." << endl;
91         return;
92     }
93     Buku* current = head;
94     cout << "Berikut buku:" << endl;
95     while (current != nullptr) {
96         cout << "ISBN: " << current->ISBN << ", Judul: " << current->Judul << ", Penulis: " << current->Penulis << endl;
97         current = current->next;
98     }
99 }
100
101 void cariBuku(int pilihan, string searchTerm) {
102     if (head == nullptr) {
103         cout << "Tidak ada buku." << endl;
104         return;
105     }
106     Buku* current = head;
107     bool ditemukan = false;
108     while (current != nullptr) {
109         if (current->Judul == searchTerm) {
110             cout << "Buku ditemukan: " << current->Judul << endl;
111             ditemukan = true;
112         }
113         current = current->next;
114     }
115     if (!ditemukan) {
116         cout << "Buku tidak ditemukan." << endl;
117     }
118 }
```

The image shows a Windows desktop with a Visual Studio Code editor open. The editor is displaying a C++ program named 'main.cpp' which implements a book search system. The program uses a switch statement to search for books based on title, author, or ISBN. The Explorer sidebar on the left shows the project structure with files like 'main.cpp' and 'main.h'. The Run and Debug console at the bottom shows the program's output, including the list of books and the search results for 'Ratu'. The top of the screen shows the Windows taskbar with various application icons and the system clock indicating 21:31 on 20/10/2025.

